Analytics

Amazon Athena

What it is:
Serverless query service to run **SQL directly on data stored in S3**

Core Features:

- No infrastructure (fully serverless)
- Uses Presto/Trino engine
- Supports CSV, JSON, Parquet, ORC
- Integrated with AWS Glue Data Catalog

When to Use:

- Ad-hoc data analysis
- Log analysis (CloudTrail, ALB logs)
- Querying large datasets in S3

Not Ideal For:

- High-frequency queries
- Low-latency applications

Exam Tips:

- Pricing = **per data scanned**
- Use **partitioning + columnar formats (Parquet)**
- Often paired with **S3 + Glue**

Amazon Kinesis Data Firehose

What it is:
Fully managed service to **deliver streaming data to destinations like S3, Redshift, OpenSearch**

Core Features:

- No shard management (fully serverless)
- Automatic scaling
- Built-in data transformation
- Direct delivery to AWS destinations

When to Use:

- Streaming data ingestion → storage
- Log collection pipelines
- Real-time data delivery

Not Ideal For:

- Custom real-time processing (use Kinesis Streams)

Exam Tips:

- Firehose = **delivery (no management)**
- If "auto delivery to S3/Redshift" → Firehose

Application Integration

Amazon EventBridge

What it is:
Serverless **event bus** for routing events between AWS services

Core Features:

- Rule-based event filtering
- Supports AWS + SaaS integrations
- Multiple targets (Lambda, SQS, Step Functions)

When to Use:

- Event-driven architectures
- Microservices communication

Not Ideal For:

- Message buffering (use SQS)

Exam Tips:

- EventBridge = **event routing + filtering**

Amazon Simple Notification Service

What it is:
Managed **publish/subscribe (push-based)** messaging service

Core Features:

- Push notifications (email, SMS, Lambda, HTTP)

- Fan-out architecture
- Topic-based messaging

When to Use:

- Sending alerts and notifications
- Broadcasting messages

Not Ideal For:

- Message storage (use SQS)

Exam Tips:

- SNS = **push model + fan-out**

Amazon Simple Queue Service

What it is:
Managed **message queue (pull-based)**

Core Features:

- Standard & FIFO queues
- Dead Letter Queue (DLQ)
- Message retention

When to Use:

- Decoupling systems
- Asynchronous processing

Not Ideal For:

- Real-time notifications

Exam Tips:

- SQS = **decoupling + buffering**

AWS Step Functions

What it is:
Serverless service to **orchestrate workflows using state machines**

Core Features:

- Visual workflow design
- Built-in retries and error handling
- Parallel execution

When to Use:

- Multi-step workflows
- Microservices orchestration

Not Ideal For:

- Simple single-step processing

Exam Tips:

- Step Functions = **workflow orchestration + retries**

Business Applications

Amazon Simple Email Service

What it is:
Cloud-based service to **send and receive emails at scale**

Core Features:

- Transactional and bulk email sending
- Email receiving support
- Integration with SNS/Lambda

When to Use:

- Sending application emails (OTP, notifications)
- Marketing emails

Not Ideal For:

- SMS notifications (use SNS)

Exam Tips:

- SES = **email service**
  If "send email notifications" → SES

Cloud Financial Management

AWS Cost and Usage Report

What it is:
Detailed report of **AWS usage and cost data**

Core Features:

- Granular billing data
- Delivered to S3
- Supports analysis with Athena

When to Use:

- Deep cost analysis
- Billing reports

Not Ideal For:

- Quick visualization (use Cost Explorer)

Exam Tips:

- CUR = **most detailed cost report**
- Often used with **Athena for analysis**

AWS Cost Explorer

What it is:
Tool to **visualize and analyze AWS costs**

Core Features:

- Cost breakdown by service
- Historical trends
- Forecasting

When to Use:

- Cost analysis
- Usage trends

Not Ideal For:

- Alerts (use Budgets)

Exam Tips:

- Cost Explorer = **visual cost analysis**

Savings Plans

What it is:
Flexible pricing model to **reduce AWS costs**

Core Features:

- Commit to usage (1 or 3 years)
- Applies to EC2, Lambda, Fargate
- Lower cost than On-Demand

When to Use:

- Predictable workloads
- Long-term cost savings

Not Ideal For:

- Short-term workloads

Exam Tips:

- Savings Plans = **cost optimization strategy**
- More flexible than Reserved Instances

Compute

Amazon EC2

What it is:
Virtual servers in the cloud to run applications

Core Features:

- Full OS control (Linux/Windows)
- Multiple instance types
- Auto Scaling + Load Balancing
- Flexible pricing (On-Demand, Reserved, Spot)

When to Use:

- Custom applications
- Long-running workloads

Not Ideal For:

- Event-driven workloads (use Lambda)

Exam Tips:

- EC2 = **full control compute**
- Spot Instances = **cost optimization**

Amazon EC2 Image Builder

What it is:
Service to **automate creation and maintenance of machine images (AMI)**

Core Features:

- Automated image pipelines
- Security patch updates
- Versioned AMIs

When to Use:

- Standardizing EC2 images
- Maintaining updated AMIs

Not Ideal For:

- Running applications

Exam Tips:

- Image Builder = **automate AMI creation**

AWS Lambda

What it is:
Serverless compute service to run code without managing servers

Core Features:

- Event-driven execution
- Auto scaling
- Pay-per-use
- Integrates with many AWS services

When to Use:

- APIs (with API Gateway)
- Automation tasks
- Event processing

Not Ideal For:

- Long-running jobs (>15 minutes)

Exam Tips:

- Lambda = **serverless + event-driven**
- Max timeout = **15 minutes**

Containers

Amazon Elastic Container Registry

What it is:
Managed service to **store Docker container images**

Core Features:

- Secure image storage (IAM integration)
- Image versioning
- High availability

When to Use:

- Store container images for ECS/EKS

Not Ideal For:

- Running containers

Exam Tips:

- ECR = **Docker image storage**

Amazon Elastic Container Service

What it is:
AWS-native service to **run and manage containers**

Core Features:

- Launch types: EC2 & Fargate
- Integrated with AWS services
- Load balancing support

When to Use:

- Microservices architecture
- Containerized applications

Not Ideal For:

- Kubernetes-specific workloads

Exam Tips:

- ECS = **easy container orchestration**
- Fargate = **serverless containers**

Amazon Elastic Kubernetes Service

What it is:
Managed service to run **Kubernetes clusters**

Core Features:

- Managed control plane
- Supports EC2 & Fargate
- High scalability

When to Use:

- Kubernetes-based applications
- Multi-cloud strategies

Not Ideal For:

- Simple container workloads

Exam Tips:

- EKS = **Kubernetes on AWS**

Database

Amazon Aurora

What it is:
High-performance **relational database** compatible with MySQL/PostgreSQL

Core Features:

- Up to 5x faster than MySQL
- Auto-scaling storage

- Multi-AZ by default

When to Use:

- High-performance relational workloads

Not Ideal For:

- NoSQL use cases

Exam Tips:

- Aurora = **high-performance RDS**

Amazon Aurora Serverless v2

What it is:
Serverless version of Aurora with **automatic scaling capacity**

Core Features:

- Auto scaling compute
- Pay per usage
- No capacity planning

When to Use:

- Variable or unpredictable workloads

Not Ideal For:

- Constant high-load systems

Exam Tips:

- Aurora Serverless = **auto-scaling DB**

Amazon DynamoDB

What it is:
Fully managed **NoSQL database**

Core Features:

- Key-value and document store
- Single-digit ms latency
- Auto scaling

- Global tables

When to Use:

- Serverless apps
- High-scale workloads

Not Ideal For:

- Complex joins

Exam Tips:

- DynamoDB = **NoSQL + serverless DB**

Amazon DynamoDB Accelerator (DAX)

What it is:
In-memory cache for DynamoDB

Core Features:

- Microsecond latency
- Fully managed
- Read-heavy optimization

When to Use:

- Improve DynamoDB read performance

Not Ideal For:

- Write-heavy workloads

Exam Tips:

- DAX = **cache for DynamoDB**

Amazon ElastiCache

What it is:
Managed **in-memory caching service (Redis/Memcached)**

Core Features:

- Ultra-fast access
- Supports Redis features

- Reduces DB load

When to Use:

- Caching
- Session storage

Not Ideal For:

- Primary database

Exam Tips:

- ElastiCache = **general caching**

Amazon RDS

What it is:
Managed relational database service

Core Features:

- Multi-AZ high availability
- Automated backups
- Read replicas

When to Use:

- Traditional relational applications

Not Ideal For:

- Massive NoSQL workloads

Exam Tips:

- Multi-AZ = **HA**, Read Replica = **scaling**

Amazon RDS Proxy

What it is:
Fully managed **database proxy for RDS/Aurora**

Core Features:

- Connection pooling
- Improves scalability

- Works with Lambda

When to Use:

- High connection workloads
- Serverless apps

Not Ideal For:

- Non-RDS databases

Exam Tips:

- RDS Proxy = **connection pooling + Lambda optimization**

Developer Tools

AWS X-Ray

What it is:
Distributed tracing service to **debug applications**

Core Features:

- End-to-end request tracing
- Service map visualization
- Latency analysis

When to Use:

- Debugging microservices
- Performance monitoring

Not Ideal For:

- Logging (use CloudWatch)

Exam Tips:

- X-Ray = **trace requests across services**